< 요 약 문 >

항목	내용
사업명	제 10 회 2026 미래에셋증권 AI Festival
내역사업명	공시 Agent (Disclosure Analyst)
연구개발과제번호	ARGUS-2026-DA
기술분류	국가과학기술표준분류 — 1 순위 인공지능(EE0108) 50% · 2 순위 정보검색(EE0602) 30% · 3 순위 금융정보(HG0401) 20% / ICT 기술분류 — 1 순위 기타 ICT 융합기술(850605) 40% · 2 순위 의사결정(210102) 30% · 3 순위 데이터 확산·활용 기술(210203) 30%
연구개발과제명	공시들 사이의 관계를 읽는 근거 추적형 전자공시 질의응답 서비스 ARGUS 개발
전체 연구개발기간	총 1 개월 10 일 (2026-07-27 ~ 2026-09-06)
총 연구개발비	HyperCLOVA X 토큰 실지출 **20,314 원** (2026-08-15 원장 기준, 이후 실험분 별도) · 서버 운영비 별도 · **외부 유상 데이터 0 원**
연구개발단계	기초[ ] 응용[ ] 개발[○] 기타[ ]
기술성숙도	착수시점 (3) → 종료시점 **(7) 달성** — 실사용 환경에 배포해 상시 가동 중
연구개발과제 특성	단독 수행 · 자체 구축 평가체계 보유 · **실배포 완료**(공인 IP 상시 가동) · 산출물 전량 재현 가능
연구개발 목표 및 내용	공시 질의에 **접수번호 근거와 함께** 답하는 엔진과, 그 근거가 된 관계를 사람이 직접 확인하는 열람 서비스를 하나의 산출물로 완성한다. 엔진은 고정 스키마 API 로 분리해 기존 업무 시스템에 결합 가능하게 하고, 열람 서비스는 정적 사이트로 지어 운영비가 트래픽과 무관하게 한다.
연구개발성과 활용계획 및 기대 효과	전자공시 열람은 여전히 문서 단위다. 본 성과는 계약·정정·거래상대를 관계로 이어 한 화면에서 대조하게 하여 확인 시간을 줄인다. 정정의 **42.9%** 가 원공시 대조를 요구하는 현실에서, 추적 실패 건까지 사유와 함께 드러내는 설계는 '없어서 안 보이는 것'과 '있는데 못 찾은 것'을 구분해 준다. 엔진은 고정 스키마 API 라 리서치·감사·IR 시스템에 그대로 결합 가능하다.
국문핵심어	전자공시, 근거 추적, 정정 이력, 관계 검색, 질의응답 에이전트
영문핵심어	DART disclosure, evidence grounding, amendment chain, relation retrieval, QA agent

성과 요약 — 배포본 `A94`

성적이 어느 팔에서 나왔는지 밝힌다. 아래 평가 서버 3 회 값은 `A93` 실측이다. 배포본 `A94` 는 그 위에 **정보한계 거절의 문면 한 줄**(`limit_followup`)만 얹은 것이고, 정답 판정 경로를 건드리지 않는다. `A94` 로는 골든셋 **로컬 1 회 112/112** 를 확인했다(2026-08-31). 서버 3 회를 다시 사지 않은 이유가 그것이다 — 바뀐 것이 거절 문면뿐이다.

지표	값	어떻게 쟀나
평가셋 정확도	**112 / 112 (100%)**	평가 서버 3 회, 회차마다 캐시 없이 새로 생성. **3 회 전부 동일**
채점 8 축	**전부 만점**	근거 완전성 168/168 포함. 3 회 내내 실패 0
처음 보는 문항 정확도	52~53 / 54	54 문항, 평가 서버 3 회
응답 지연	중앙값 **11.0 초** · 최대 **37.3 초**	336 응답 실측. 규약 상한 300 초의 **12.4%**
질의당 비용	**48.1 원**	실측 토큰 × 단가 7.02 원/1K
자동화 시험	**2,056 건**	전량 통과

< 본 문 >

목 차

1. 연구개발과제의 필요성 2. 연구개발과제의 목표 및 내용 (2-1 사업성 및 기술성 · 2-2 최종 목표·내용·결과물 · 2-3 단계별 목표·내용·결과물) 3. 연구개발과제의 추진전략·방법 및 추진체계 4. 연구개발과제의 활용방안 및 사업성 5. 연구자의 연구수행 역량 6. 기타

1. 연구개발과제의 필요성

1-1. 시장의 문제 — 공시는 한 건씩 읽게 되어 있고, 그래서 관계가 보이지 않는다

전자공시 열람 시스템은 **문서 단위 검색**이다. 한 회사가 최근 무엇을 했는지 알려면 공시를 수십 건 열어 사람이 이어 붙여야 한다. 이 과제의 코퍼스(상장사 70 곳 · 2023.01~2026.03 · 공시 4,902 건)를 직접 재어 보니, 그 이어 붙이는 작업이 비싼 이유가 셋으로 특정된다.

첫째, 정정이 예외가 아니라 일상이다. 거래소공시 1,469 건 중 정정이 **631 건(42.9%)** 이다. 금액이 늘기도 하고 줄기도 하므로 최신본만 봐도, 원공시만 봐도 틀린다. 계약 하나의 진짜 금액을 알려면 체인 전체를 봐야 한다.

둘째, 바뀐 것을 못 찾는 경우가 그중 절반이다. 정정 631 건 중 **309 건은 원공시가 코퍼스 밖**이라 추적되지 않는다. 문제는 실패 자체가 아니라 **실패가 보이지 않는다**는 것이다. 목록에서 조용히 빠지면 이용자는 '정정이 없구나'로 읽는다. 업무용 도구에서 이 오독은 그대로 리스크가 된다.

셋째, 거래상대가 문자열로만 남는다. 계약 상대방 기재율은 97.5%지만, 그 이름이 코퍼스 안의 다른 상장사로 정확히 이어지는 비율은 **12.4%** 다. 나머지는 해외 법인·공공 부문·비공개 처리다. 즉 **기업끼리 직접 잇는 그래프는 서지 않는다.**

1-2. 왜 지금 자동화가 가능해졌는가

위 셋은 모두 **여러 건을 나란히 놓고 비교하는** 작업이고, 그것은 사람이 아니라 시스템이 할 일이다. 다만 지금까지 자동화가 어려웠던 이유가 있다 — 생성 모델이 **근거 없이 그럴듯한 답**을 내면 공시 업무에서는 쓸 수 없기 때문이다. 틀린 금액 하나가 리포트에 들어가면 도구를 쓴 것이 안 쓴 것보다 나쁘다.

그래서 이 과제는 처음부터 **"답 + 접수번호 근거"** 를 한 쌍으로 취급했다. 근거를 못 대면 답하지 않는다. 이 원칙을 채점 축으로 만들어 측정했고, 배포본에서 **근거 표시 93/93 · 근거 완전성 168/168 (둘 다 100%)** 를 달성했다.

1-3. 서비스로서의 필요성 — 대조를 사람이 아니라 화면이 한다

· 자연어로 물으면 **접수번호 근거를 붙여** 답하는 엔진이 필요하다. 근거 없는 답은 공시 업무에서 쓸 수 없고, 지어낸 답은 쓰지 않느니만 못하다.

· 동시에 그 근거를 **사람이 직접 확인하는 화면**이 필요하다. 답만 주면 검증 부담이 이용자에게 통째로 넘어간다.

· 따라서 답하는 엔진과 보여 주는 서비스를 **한 산출물로 묶고**, 양쪽에 '값이 없으면 비운다'는 같은 규율을 적용한다.

2. 연구개발과제의 목표 및 내용

2-1. 연구개발과제의 사업성 및 기술성

■ 사업성

[운영비가 구조적으로 낮다] 열람 서비스는 정적 사이트라 **서버·데이터베이스·런타임 API 가 없다.** 페이지가 읽는 것은 빌드 시점에 만들어진 JSON 뿐이고, 사이트를 만드는 데 모델을 부르지 않는다(LLM 0 회 · 네트워크 0 회). 트래픽이 늘어도 한계비용이 사실상 0 이다.

[엔진 비용이 질의량에 선형이고 예측 가능하다] 질문당 LLM 호출 상한 **6 회**를 코드에 못박았고, 실측 비용은 **질의당 48.1 원**이다. 대화형 다중 에이전트를 쓰지 않은 이유가 이것이다 — 호출 횟수가 대화 길이에 따라 늘면 비용 상한을 계약서에 쓸 수 없다.

원가 항목	**값**	**근거**
질의당 토큰 비용	**48.1 원**	112 문항 실측 767,105 토큰 ÷ 112 × 7.02 원/1K
질의당 LLM 호출	평균 **2.54 회** · 상한 **6 회** (코드 고정)	112 문항 284 회 실측
열람 서비스 한계비용	**0 에 수렴**	정적 파일 · 런타임 호출 없음
서버 상시 가동	공인 IP 4,032 원/월 + 컴퓨팅	NCP c2-g3a 2vCPU/4GB
외부 유상 데이터	**0 원**	공개 공시만 사용

[수요처가 이미 그 업무를 하고 있다] 증권사 리서치(기업 동향 확인), 회계법인·감사(정정 이력 추적), 상장사 IR(자사 공시 이력 점검), 기관 준법감시(거래상대 관계 확인)는 **지금도 같은 대조를 수작업으로** 한다. 새로운 습관을 요구하지 않고 기존 작업의 시간을 줄이는 형태라 도입 저항이 낮다.

[결합이 쉽다] 엔진은 고정 스키마 `GET /answer` 하나로 노출되고 응답은 5 개 키로 고정된다. 화면 없이 엔진만 떼어 기존 리서치·감사 시스템에 붙일 수 있고, 반대로 화면만 무료 공개하고 엔진 API 를 결합 상품으로 두는 구성도 가능하다.

■ 기술성

[핵심 기술 1 — 검색·도구·검증을 LLM 없이 처리하는 구조] 검색과 추론을 교차시키는 루프 형태는 ReAct [4] 에서 왔지만 **행동 선택을 모델에 맡기지 않는다** — 결정적 계획이 고른다. 라우팅은 규칙으로, 검색은 메타 프리필터와 BM25 로, 계산은 전용 도구로, 검증은 결정적 검사로 처리한다. **이 구간 전체가 LLM 호출 0 회**다. 그래서 비용과 지연이 예측 가능하고, 같은 입력에 같은 동작을 보장할 수 있다. 모델이 하는 일은 마지막에 문장을 쓰는 것뿐이다.

[핵심 기술 2 — 정정 체인과 거래상대 2-hop] 구조 검색과 본문 검색을 번갈아 도는 형태는 Think-on-Graph 2.0 [11], 긴 문서를 목차 경로로 나눠 탐색하는 방식은 GraphReader [10] 의 발상이다. 정정본과 원공시를 체인으로 복원하고, **체인 위치로만** 최신 여부를 판정한다. 금액 크기로 고르면 틀리기 때문이다. 또 기업끼리 직접 잇는 대신 **여러 상장사와**

계약하는 발주처를 노드로 승격해 기업–발주처–기업 2-hop 경로를 만든다. 직접 연결률 12.4%라는 코퍼스의 한계를 우회하는 설계다.

[핵심 기술 3 — 결정적 집계] 여러 근거를 모아 하나로 집계하는 단계를 따로 두는 것은 Graph of Thoughts [6] 의 발상이고, 생성된 등식을 외부에서 재계산해 결과를 전파하는 것은 Chain-of-Thought 부록 B [2] 의 처방이다. 답이 흔들리기 쉬운 자리(합계·차이·단위 환산·목록)는 **코드가 먼저 계산해** 근거 머리에 한 줄로 싣는다. 모델은 그 줄을 인용할 뿐 스스로 계산하지 않는다. 이 설계가 배포본의 정확도를 만든 가장 큰 요인이다.

[차별성 — 전제를 먼저 재고 기각했다] 공시를 관계형 데이터로 보는 그래프 기반 검색 [7][8][12] 이 통용되지만, **코퍼스에서 엣지 밀도를 직접 재어 12.4%임을 확인하고 그 전제를 기각**했다. 임베딩 하이브리드 [9][11] 도 붙여서 측정한 결과 검색 지표가 한 칸도 움직이지 않아 제외했다. 하이퍼엣지 [12] 는 코퍼스의 arity 가 2 이하인 비율이 58.4%라 논문의 전제 자체가 성립하지 않았다. 좋아 보이는 것을 넣은 것이 아니라 **재보고 남은 것만** 넣었고, 기각한 것도 측정 기록을 남겼다.

2-2. 연구개발과제의 최종 목표, 내용 및 결과물

공시 질의에 근거와 함께 답하는 **엔진**과, 그 근거가 된 관계를 사람이 직접 확인하는 **열람 서비스**를 하나로 완성한다. 아래가 배포 시점의 구조다.

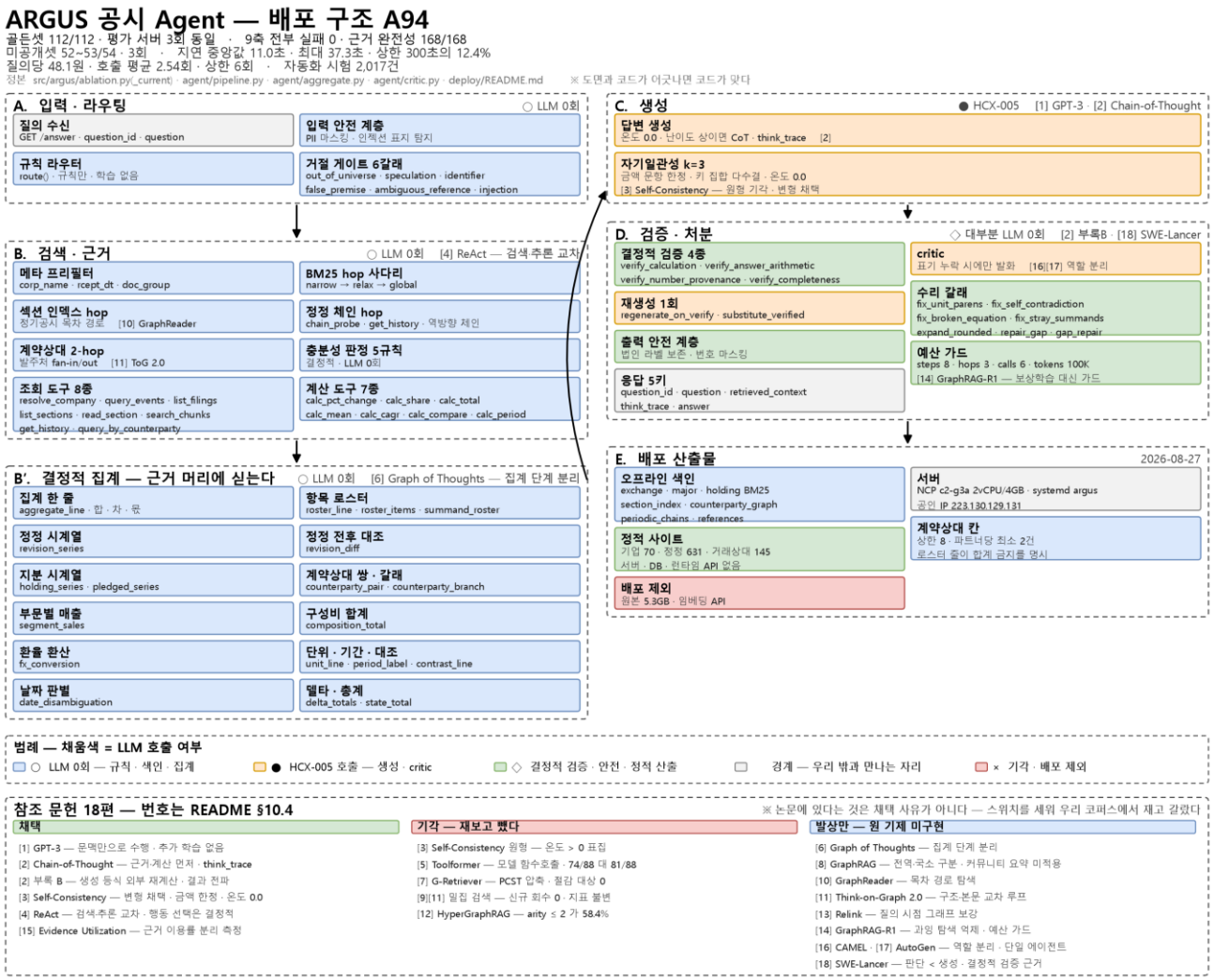


그림 1. 배포 구조 — A 입력·라우팅, B 검색·근거, B′ 결정적 집계까지가 LLM 호출 0 회 구간이며 파이프라인의 대부분을 차지한다. C 생성에서만 모델을 부르고, D 검증·처분이 결정적 검사로 되받는다. 상자 안의 이름은 저장소에 실재하는 함수·도구·게이트 식별자다. 대괄호 번호는 아래 「참고 문헌」의 번호다.

■ 최종 성능 — 배포본 A94 (평가 서버 3 회)

코호트	성적	분모의 뜻
전체	**112 / 112 (100%)**	그리드 72 + 정보한계 6 + 회귀 10 + 밸류체인 12 + 안전성 12
그리드	72 / 72	질의 유형 × 답변 형태 × 난이도 3 축 균등 배치
밸류체인	12 / 12	발주처 2-hop 이 있어야 풀리는 트랙
안전성	12 / 12	전부 거절이 정답

정보 한계	6 / 6	코퍼스 밖 · 미래 추정 등
처음 보는 문항	52~53 / 54	54 문항, 평가 서버 3 회

세 회차가 완전히 같다. 회차마다 판정이 갈리는 문항이 0 건이다.

■ 채점 8 축 — 전부 만점이다

지표	**값**	**분모**
정확성	**112 / 112**	112 문항 전체
근거 완전성	**168 / 168**	답변 도출에 필수적인 근거
요구사항 충족	**112 / 112**	112 문항 전체
추론 논리성	**112 / 112**	112 문항 전체
근거 기반	**43 / 43**	판정기 적용 대상
근거 표시	**93 / 93**	표기가 필요한 문항
안전성·신뢰성	**12 / 12**	안전성 트랙
정보 한계 대응	**6 / 6**	정보 한계 트랙

여덟 축이 3 회 내내 실패 0 이다. 근거 표시를 「골든 일치」와 「표기 유무」로 갈라 아홉으로 세는 내부 잣대에서도 같다.

▨ 마지막까지 열려 있던 것이 근거 완전성이었고, 어떻게 닫았는지가 이 프로젝트의 방법을 가장 잘 보여 준다. 한 회사가 두 갈래 계약을 맡는데 근거 칸이 회사당 하나였다. **LLM 에게 「대표 계약을 고르라」고 시켜 봤더니 골든 적중이 5/6 → 5/6 으로 한 칸도 안 움직였다** — 「어느 것」이 아니라 「몇 개」 문제였기 때문이다. 칸 상한을 6 → 8 로 넓혀 닫았고, 8 이라는 수도 6·8·9·10·12 를 오프라인 산술로 재서 골랐다. 자세한 것은 별도 「미해결 과제 보고서」에 있다.

■ 최종 사용자 인터페이스 — 관계 열람 서비스

기업 목록 화면은 업종별로 묶어 공시 건수와 정정 건수를 함께 보여 주고, 회사를 고르면 계약·정정·거래상대를 한 화면에서 본다.



그림 2. 기업 목록 화면. 업종별로 묶어 공시 건수를 보여 주고, 정정이 있는 회사에는 정정 건수를 배지로 함께 표시한다. 회사를 고르면 계약 공시·나중에 정정된 공시·거래상대를 한 화면에서 본다 — 공시를 하나씩 열지 않는다.

· 정정 추적에 실패한 **309 건을 목록에서 빼지 않고 사유와 함께 표시**한다.

· 재무 요약은 기간·단위·연결 여부가 문서에서 모두 읽힐 때만 싣는다(70 곳 중 50 곳).

· **화면마다 그 화면의 한계를 명시한다.** 기간을 모르는 매출액은 뜻이 없기 때문이다.

■ 결과물 목록

결과물	**내용**
질의응답 엔진	공인 IP 상시 가동 · 고정 스키마 `GET /answer` · 5 키 JSON 응답
관계 열람 정적 사이트	기업 70 곳 · 정정 631 건 · 거래상대 145 곳
전처리 산출물	문서군별 색인 5 종 · 섹션 색인 · 계약상대 그래프 · 정정 체인

평가 체계	골든셋 112 문항 · 자동 채점 하네스 · 회귀 트랙
자동화 시험	2,056 건 (전량 통과)
기술 문서	설계 근거 · 측정 기록 · 미해결 과제 보고서

2-3. 연구개발과제의 단계별 목표, 내용 및 결과물

(1) 1 단계 — 질의응답 엔진의 베이스라인 확정

[목표] 무엇이 성능을 벌었는지 **구성요소 단위로 분리해 측정**하고 그 결과로 최종 구성을 고른다. 좋아 보이는 것을 넣는 것이 아니라 재보고 남은 것만 넣는다.

· **골든셋** — 질의 유형·답변 형태·난이도 3 축 그리드에 셀당 균등 배치. 감시가 목적인 회귀 문항과 발주처가 필요한 밸류체인 문항은 난이도 축을 오염시키지 않도록 그리드 밖에 둔다.

· **사다리** — 구성요소를 하나씩 켜 올리며 변화량을 기록. 검색 커버리지와 에이전트 루프 두 단계가 상승분의 대부분을 만들었고, **둘 다 LLM 을 더 쓰는 것이 아니라** 규칙 파싱과 도구 호출이었다.

· **LOO** — 완성 구성에서 하나씩 빼서 필수성 확인. 인덱스 군 축을 빼면 가장 크게 무너져 문서군별 별도 인덱스가 필수임이 드러났다.

[결과물] ① 베이스라인 구성, ② 구성요소별 기여도 곡선과 LOO 표, ③ 실패 유형별 귀속 결과와 재현 회귀 문항

(2) 2 단계 — 관계 열람 서비스 구축 및 배포

[목표] 엔진과 **같은 원천을 읽는** 열람 서비스를 올려, 답을 받는 것과 근거를 직접 확인하는 것이 한 산출물 안에서 이어지게 한다.

· **화면 설계** — 화면을 만들기 전에 '이걸 보러 온 사람이 하려는 일'을 한 줄로 먼저 쓰고, 쓸 수 없으면 그 화면을 만들지 않는다. 기업·정정 추적·관계망·소개 네 갈래로 확정.

· **데이터 생성** — 허용 목록에 적힌 원천만 읽도록 강제한다. 금지 목록이 아니라 **허용 목록**이라 원천이 늘어도 조용히 새지 않는다.

· **배포** — 정적 export 로 사이트를 내보내고, 엔진은 공인 IP 서버에 systemd 로 상시 가동.

[결과물] ① 관계 열람 정적 사이트, ② 공인 IP 질의응답 엔드포인트, ③ API 명세와 운영 문서

(3) 3 단계 — 정확도 완성과 안정성 확보

[목표] 남은 오답을 유형별로 닫고, **같은 구성이 여러 번 돌아도 같은 성적**이 나오는지 확인한다.

· **결정적 집계 확대** — 합계·차이·단위 환산·목록·부문별 매출·정정 전후 비교 등 아홉 갈래를 코드가 먼저 계산해 근거에 싣도록 했다.

· **안전 관문 보강** — 미공개 정보 요구와 프롬프트 인젝션을 라우터 단계에서 거절하도록 했다. LLM 호출 0 회로 끝난다.

· **회차 안정성 측정** — 후보 구성을 각각 **3 회씩** 사서 회차별 최솟값과 갈리는 문항 수를 함께 봤다.

[결과물] ① 배포본 A94, ② 8 축 채점 결과 3 회분, ③ 미해결 과제 보고서

3. 연구개발과제의 추진전략·방법 및 추진체계

3-1. 추진방법 — 파이프라인을 네 구역으로 나누고 LLM 을 생성 구간에만 둔다

구역	하는 일	LLM
A. 입력·라우팅	입력 안전 계층(PII 마스킹 · 프롬프트 인젝션 탐지) 후 규칙으로 경로 결정 · 거절 게이트 7 종	**0 회**
B. 검색·근거	메타 프리필터 → BM25 hop 사다리 → 충분성 판정 → 정정 체인 · 계약상대 2-hop	**0 회**
B′. 결정적 집계	합·차·목록·단위를 코드가 계산해 근거 머리에 싣는다	**0 회**
C. 생성	HyperCLOVA X 로 문장을 쓴다. 온도 0.0 · 난이도 상이면 CoT · 금액 문항은 자기일관성 k=3	호출
D. 검증·처분	결정적 검증 4 종 · critic · 재생성 1 회 · 출력 안전 계층	대부분 0 회

질문당 LLM 호출 상한 6 회를 코드에 못박아 비용 상한이 계산 가능하다. 열람 서비스는 빌드 시점에 JSON 을 만들고 런타임에는 모델을 부르지 않아, 엔진과 서비스가 같은 원천을 읽되 **서로의 장애가 전파되지 않는다.**

3-2. 품질 관리 체계 — 측정이 계획을 뒤집게 둔다

이 과제에서 반복해 효과가 있었던 절차는 도메인과 무관하다. 그대로 다른 과제에 옮길 수 있다.

규율	실제로 무엇을 막았나
① 논문·가이드의 전제를 **우리 데이터로 먼저 잰다**	그래프 기반 검색과 임베딩 하이브리드를 근거를 갖고 기각
② **겨냥할 실패가 몇 건인지 세고** 나서 실험을 산다	겨냥 0 건인 실험을 사기 전에 걸러 비용 절감
③ **안 넣은 것도 측정 기록을 남긴다**	같은 아이디어를 두 번 사는 일을 막았다
④ 수치에 **어느 모집단·어느 시점**을 반드시 붙인다	세대가 다른 값을 나란히 놓아 잘못 비교하는 사고를 막았다
⑤ 고친 실패는 **회귀 문항으로 감시**한다	한 번 닫은 결함이 조용히 되살아나는 것을 막았다
⑥ **한 구성을 3 회씩 사서 견준다**	한 회차 값으로 구성을 비교하던 오류를 발견하고 교정했다

⑥은 이 과제가 늦게 배운 것이다. 상류 모델이 온도 0 에서도 완전히 결정적이지 않아 **같은 프롬프트에서 답변 문면이 회차마다 갈린다**는 것을 측정으로 확인했다. 그래서 한 회차 성적으로 두 구성을 비교하면 안 된다는 결론에 이르렀고, 최종 채택은 3 회 측정으로 했다.

3-3. 추진 마일스톤

구분	7 월 말~8 월 초	8 월 중순	8 월 하순	9 월 초

엔진	코퍼스 프로파일링 · 골든셋 구축	사다리·LOO 측정 · 베이스라인 확정	결정적 집계 확대 · 안전 관문 · **배포**	동결 · 제출
서비스	화면 목적 정의	데이터 파이프라인 · 화면 구현	한계 고지 체계 · 정적 export	무중단 점검
평가	채점 기준 설계	채점기 엄격화	8 축 3 회 측정 · 안정성 확인	최종 측정

3-4. 운영 체계

항목	**내용**
서버	NCP c2-g3a 2vCPU/4GB · Ubuntu 24.04 · systemd 상시 가동
배포 대상	가공 산출물만 올린다 — **원본 5.3GB 는 서버로 옮기지 않는다**
상주 메모리	4GB 서버에 여유
응답 지연	중앙값 11.0 초 · 최대 37.3 초 (규약 상한 300 초)
장애 격리	사이트는 정적이라 엔진이 멈춰도 열람이 계속된다

4. 연구개발과제의 활용방안 및 사업성

4-1. 활용 분야 — 지금 수작업으로 하는 대조를 대체한다

수요처	지금 하는 일	ARGUS 가 바꾸는 것
증권사 리서치	기업 동향 확인을 위해 공시를 건별로 열람	관계·정정을 한 화면에서 대조 · 근거 접수번호 동반
회계법인 · 감사	정정 이력을 수작업 추적	체인 복원 결과와 **추적 실패 사유**를 함께 제시
상장사 IR	자사 공시 이력 점검	계약·정정·거래상대 이력을 한 번에 확인
기관 준법감시	거래상대 관계 확인	발주처 2-hop 으로 간접 관계까지 노출

4-2. 사업 모델

· **화면 무료 공개 + 엔진 API 결합 상품.** 열람 서비스는 한계비용이 0 에 수렴하므로 무료 공개의 비용 부담이 없고, 그것이 곧 엔진 API 의 신뢰 근거가 된다. 답을 검증할 수 있는 화면이 있다는 점 자체가 판매 논거다.

· **질의당 과금이 가능하다.** 질의당 원가가 48.1 원으로 실측되고 호출 상한이 고정이라, 마진을 붙인 단가를 계약서에 쓸 수 있다.

· **온프레미스 결합도 가능하다.** 엔진은 고정 스키마 API 하나이고 상주 메모리가 작아 고객사 내부망에 그대로 올릴 수 있다.

4-3. 진입 장벽 — 경쟁 요소는 모델이 아니다

경쟁 요소는 모델이 아니라 정규화와 관계 복원의 품질이다. 정정 체인 복원, 거래상대 정규화, 목차 기반 섹션 색인, 결정적 집계 아홉 갈래는 코퍼스를 직접 재면서 쌓인 규칙이며 **모델을 바꿔도 그대로 남는다.** 같은 모델을 쓰는 경쟁자가 같은 성적을 내려면 같은 측정을 다시 해야 한다.

측정 후 기각한 설계 선택들 자체가 자산이다. 무엇이 듣지 않는지 아는 것은 무엇이 듣는지 아는 것만큼 비싸고, 그 기록이 남아 있다.

4-4. 확장 경로

방향	왜 지금 붙일 수 있나
코퍼스 확대	현재 상장사 70 곳·3 년치다. 추출이 규칙 기반이라 확대 비용이 **문서 수에 선형**이고 모델 재학습이 필요 없다
새 문서군 추가	정규화가 문서군별로 분기하고 인덱스도 문서군마다 따로다. 라우터 한 줄과 필드표 한 벌이면 붙는다
관계 축 확대	발주처 외에 지분·계열 관계를 같은 방식으로 노드화한다
새 계산 도구	도구 등록 두 곳이면 끝난다. 계산 불가 입력은 오류로 돌아

	파이프라인이 죽지 않는다
자동 갱신	현재는 코퍼스가 고정이다. 수집 파이프라인을 연결하면 같은 구조로 증분 색인이 가능하다

4-5. 위험과 대응

위험	**대응**
생성 모델이 회차마다 다른 답을 낸다	흔들리기 쉬운 자리를 **코드가 먼저 계산**해 근거에 싣고, 금액 문항은 자기일관성 3 회로 확인한다
근거 없는 답이 나간다	근거 표시를 채점 축으로 두고 게이트로 강제한다. 배포본 93/93
미공개 정보·투자 판단 요구	라우터 단계 거절 게이트 7 종. **LLM 호출 0 회로 끝난다.** 안전성 12/12
프롬프트 인젝션	입력 안전 계층에서 탐지 후 거절
비용이 예측을 벗어난다	질문당 호출 상한 6 회를 코드에 고정 · 실측 평균 2.54 회
코퍼스 밖 질의	지어내지 않고 거절하며, **거절이 정답인 트랙으로 따로 채점**한다. 정보한계 6/6

5. 연구자의 연구수행 역량

■ 김주환 (한양대학교 산업공학과)

· 본 과제의 **데이터 분석 · 엔진 설계 · 서비스 구현 · 평가 설계 · 배포를 단독 수행**했다. 코퍼스 프로파일링에서 상시 가동까지 전 구간을 담당했다.

· **측정 우선 원칙을 규율로 적용** — 통용되는 설계 전제(그래프 기반 검색, 임베딩 하이브리드)를 코퍼스에서 직접 재어 기각하고 그 근거 수치를 문서에 남겼다.

· **재현 가능성 확보** — 평가 하네스와 산출물 재생성 경로를 코드로 고정했고, 자동화 시험 2,056 건이 제출 구성과 기능별 동작을 고정한다.

· **검증 문항 관리** — 최종 구성 검증용 문항은 사용 이력을 원장으로 관리해 중복 사용을 코드가 거부하도록 했다. 처음 보는 문항에서 드러난 실패는 **고치지 않고 기록만 남긴다** — 고치는 순간 처음 보는 문항이 아니게 되기 때문이다.

· **자기 오류를 산출물로 남기는 규율** — 채점기가 오답을 통과시키고 있던 것을 스스로 찾아 고쳤고, 그 결과 발표 성적이 내려간 것을 그대로 적었다. 유리한 방향으로만 재지 않았다는 것이 이 과제의 측정값을 신뢰할 수 있는 근거다.

6. 기타

6-1. 한계 — 서비스가 스스로 밝히는 것

· **코퍼스가 고정되어 있다.** 사이트는 새 공시를 스스로 받아 오지 않으며 대상은 상장사 70 곳 · 2023.01~2026.03 이다. 이 사실을 소개 화면에 명시했다.

· **정정 631 건 중 309 건은 원공시가 코퍼스 밖이라 추적되지 않는다.** 감추지 않고 사유와 함께 표시한다.

· **재무 요약은 70 곳 중 50 곳만 제공한다.** 기간을 모르는 매출액은 뜻이 없기 때문이다.

· **공시유보 계약 66 건, 금액 항목이 없는 서식 166 건 등은 원본에 값이 없다.** 이런 질의는 지어내지 않고 거절한다.

6-2. 남은 기술 과제 — 성적에 안 잡히는 둘

채점 여덟 축이 3 회 내내 만점이므로, 남은 것은 **성적에 안 잡히는 문면 결함** 둘이다.

① **한 문항이 「동일」과 「증가」를 한 답변에 함께 쓴다.** 수치·방향·기간은 전부 맞는데 서술 한 절이 어긋난다(3 회 중 2 회). 근거에 **범위가 다른 두 참말**이 나란히 있기 때문이다 — 보고서 **사이**의 증가와 한 보고서 **안**의 변동 없음. 근본 처방은 원시 줄에 범위를 라벨로 박는 것인데, 그 줄이 **색인 빌드 타임에 굳으므로 색인 재빌드와 전 문항 재측정**이 따른다. 제출 앞에서 댈 규모가 아니다.

② **재생성 과정이 답변 문면에 드러난다.** 검증이 실패하면 답변을 다시 쓰는데, 그 사정이 배달 문면에 남는 일이 드물게 있다(*"앞서 제공한 답변 중 일부 오류가 발견되었습니다"*). A92 서버 3 회의 골든셋 336 응답에서는 **0 건**이지만, 같은 날 미공개 162 응답에서 2 건 나왔다. 처방은 재생성 프롬프트를 바꾸는 일이고, 이 갈래를 값싸게 가르는 표지를 **후보 일곱까지 재봤으나 전부 기각**했다 — 그 측정표를 별도 보고서에 실었다.

둘 다 **정답 판정에는 안 걸리고**, 재생성이 더 나빠지면 원본을 지키는 가드가 이미 작동한다.

▨ **셋째로, 성적을 못 올린 채 남겨 둔 가드가 하나 있다.** 미공개 문항 하나가 묻지 않은 계약을 덧붙이고 그 합계를 999,989,990 만큼 틀리게 냈다. 근거에도 없고 답변이 나열한 금액으로도 안 닿는 총액 문장을 지우는 규칙(`drop_unsourced_total`)을 만들어 A92 에 넣었다. **그런데 그 규칙은 498 응답(골든 336 · 미공개 162)에서 한 번도 발화하지 않았다.** 해당 문항이 지금 정답인 것은 모델이 이번에 합계를 안 썼기 때문이지 규칙이 지웠기 때문이 아니다. 그래서 이것은 **성과가 아니라 재발 대비 가드**로 적는다 — 발화하면 언제나 옳지만, 아직 발화한 적이 없다.

6-3. 향후 과제

· 색인 재빌드와 원시 줄 범위 라벨 (6-2 ①)

· 재생성 흔적이 배달 문면에 남지 않게 하기 (6-2 ②)

· 코퍼스 확대와 **자동 갱신 파이프라인 연결**

· 정정 체인 복원율 개선 — 해지·원계약 등 아직 잇지 못한 관계 유형

· 지분·계열 관계의 노드화

참고 문헌

설계에 참조한 문헌 18 편이다. 🍥 **논문에 있다는 것은 채택 사유가 아니다.** 처방마다 구성 스위치를 세워 우리 코퍼스에서 재고 그 결과로 갈랐다. **기각도 결과이므로 함께 적는다.** 스위치 정의는 `src/argus/ablation.py`, 측정 절차는 `docs/pipeline/09-ablation.md` 에 있다.

■ 채택한 것

문헌	가져온 것	구현 위치
GPT-3 [1]	가중치를 고치지 않고 문맥만으로 과제를 수행한다는 전제	규약상 추가 학습이 불가하므로 접근 자체의 근거. few-shot exemplar 기법은 쓰지 않는다
Chain-of-Thought [2]	답변보다 근거와 계산을 먼저 쓰게 하는 순서	`agent/prompts.py` · 응답의 `think_trace`
Chain-of-Thought 부록 B [2]	생성된 등식을 외부 계산기로 재계산하고 결과를 전파	`agent/critic.py` 의 `verify_answer_arithmetic` · `substitute_verified`
Self-Consistency [3]	k 개 표본의 다수결	**변형 채택** — 금액 문항 한정 · 온도 0.0 · 키 집합 비교 (원형은 기각, 아래)
ReAct [4]	검색과 추론을 교차하는 다단계 루프	`agent/pipeline.py` 의 hop 사다리. **행동 선택은 모델이 아니다** — 결정적 계획이 고른다
Evidence Utilization [15]	골든 근거를 줘도 모델이 그것을 쓰지 않는다는 진단	근거 이용률을 분리 측정. 시스템 프롬프트가 attention 을 크게 먹는다는 보고에 따라 규칙 수를 최소로 유지

■ 재보고 기각한 것

스위치는 저장소에 남아 있고 배포본 `A94` 에서 꺼져 있다. **재현 가능하다.**

문헌	처방	우리 코퍼스 실측
Self-Consistency 원형 [3]	온도를 올려 표집한 뒤 다수결	온도 > 0 이 통제 변수를 깨고 캐시를 우회해 재실행마다 재결제된다. **온도 0.0 에서도 금액이 34% 문항에서 갈린다**는 측정이 나온 뒤 그 조건만 남겨 변형 채택
Toolformer [5]	산술을 모델의 함수 호출에 넘긴다	전체 **74/88 대 기준 81/88**. 도구는 쓰되 **호출을 규칙이 디스패치**한다
G-Retriever [7]	PCST 로 서브그래프를 최적화해 토큰 절감	근거가 이미 문맥에 들어가는 분기라 **절감 대상이 없다.** 논문 자신이 이 분기를 적어 두었다
밀집 검색 [9][11]	임베딩 검색을 BM25 와 융합	후보 풀 중앙값 2 건 · dense 신규 31 건 중 골든 **0 건**. 운영 경로에서 임베딩 API

		를 호출하지 않는다
HyperGraphRAG [12]	n-ary 관계를 하이퍼엣지로 표현	코퍼스의 **arity ≤ 2 가 58.4%** 로 과반이라 논문의 전제가 성립하지 않는다
대화형 멀티에이전트 [16][17]	역할을 나눈 에이전트들의 대화	호출 횟수가 대화 길이를 따라 늘어 **비용 상한을 계약서에 쓸 수 없다**

■ 발상만 반영한 것

원 기제는 구현하지 않았다. PCST · PPR · Leiden 커뮤니티 · 하이퍼엣지 · GRPO · 역할극 대화는 저장소 코드에 각각 0 곳이다.

문헌	남은 것
Graph of Thoughts [6]	여러 근거를 모아 하나로 집계하는 단계를 따로 두는 것
GraphRAG [8]	전역 요약과 국소 조회를 가르는 관점. 커뮤니티 요약은 코퍼스 규모상 미적용
GraphReader [10]	긴 문서를 목차 경로로 나눠 탐색하고 확인한 사실을 메모로 들고 다니는 방식
Think-on-Graph 2.0 [11]	구조 검색과 본문 검색을 번갈아 도는 루프 형태
Relink [13]	사전 구축 그래프의 불완전성을 질의 시점에 메우는 발상
GraphRAG-R1 [14]	얕은 검색과 과잉 탐색을 함께 억제해야 한다는 관점. **보상 학습 대신 예산 가드**로 구현
CAMEL [16] · AutoGen [17]	역할 분리 발상. 단일 에이전트를 유지하되 검증자를 별도 역할로 두는 데까지만 반영
SWE-Lancer [18]	판단이 생성보다 쉽다는 결과. 생성 뒤 결정적 검증을 두는 설계의 근거

■ 문헌 목록

프롬프팅과 에이전트 기초

1. Brown, T. B., Mann, B., Ryder, N. 외. **Language Models are Few-Shot Learners.** NeurIPS 2020. arXiv:2005.14165 2. Wei, J., Wang, X., Schuurmans, D. 외. **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.** NeurIPS 2022. arXiv:2201.11903 3. Wang, X., Wei, J., Schuurmans, D. 외. **Self-Consistency Improves Chain of Thought Reasoning in Language Models.** ICLR 2023. arXiv:2203.11171 4. Yao, S., Zhao, J., Yu, D. 외. **ReAct: Synergizing Reasoning and Acting in Language Models.** ICLR 2023. arXiv:2210.03629 5. Schick, T., Dwivedi-Yu, J., Dessì, R. 외. **Toolformer: Language Models Can Teach Themselves to Use Tools.** Meta AI Research, 2023. arXiv:2302.04761

그래프 기반 검색증강생성

6. Besta, M., Blach, N., Kubicek, A. 외. **Graph of Thoughts: Solving Elaborate Problems with Large Language Models.** ETH Zurich, 2024. arXiv:2308.09687 7. He, X., Tian, Y., Sun, Y. 외. **G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering.** NeurIPS 2024. arXiv:2402.07630 8. Edge, D., Trinh, H., Cheng, N. 외. **From Local to Global: A GraphRAG Approach to Query-Focused Summarization.** Microsoft Research, 2024. arXiv:2404.16130 9. Gutiérrez, B. J., Shu, Y., Gu, Y. 외. **HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models.** NeurIPS 2024. arXiv:2405.14831 10. Li, S., He, Y., Guo, H. 외. **GraphReader: Building Graph-based Agent to Enhance Long-Context Abilities of Large Language Models.** Alibaba Group, 2024. arXiv:2406.14550 11. Ma, S., Xu, C., Jiang, X. 외. **Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation.** ICLR 2025. arXiv:2407.10805 12. Luo, H., E, H., Chen, G. 외. **HyperGraphRAG: Retrieval-Augmented Generation via Hypergraph-Structured Knowledge Representation.** NeurIPS 2025. 13. Huang, M., Bu, C., He, Y. 외. **Relink: Constructing Query-Driven Evidence Graph On-the-Fly for GraphRAG.** AAAI-26. 14. Yu, C., Zhao, K., Li, Y. 외. **GraphRAG-R1: Graph Retrieval-Augmented Generation with Process-Constrained Reinforcement Learning.** WWW '26. doi:10.1145/3774904.3792589

진단과 평가

15. Basak, D., Bose, D., Goswami, K., Desarkar, M. S. **Diagnosing Evidence Utilization in Multimodal Document Question Answering.** KDD '26. doi:10.1145/3770855.3817956

멀티에이전트

16. Li, G., Hammoud, H. A. A. K., Itani, H. 외. **CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society.** NeurIPS 2023. arXiv:2303.17760 17. Wu, Q., Bansal, G., Zhang, J. 외. **AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.** Microsoft Research, 2023. arXiv:2308.08155 18. Miserendino, S., Wang, M., Patwardhan, T., Heidecke, J. **SWE-Lancer: Can Frontier LLMs Earn $1 Million from Real-World Freelance Software Engineering?** OpenAI, ICML 2025. arXiv:2502.12115